Cell-Wise Self-Optimization: Making Pretrained Model Better in Remote Sensing Counting

Shengqin Jiang, Qian Jie, Fengna Cheng, Haokui Zhang, Yu Liu, and Qingshan Liu, *Senior Member, IEEE*

***Abstract*—Recently, remote sensing counting has drawn a lot of attention due to its wide application requirements. However, most existing approaches tend to focus on optimizing the network backend or designing new loss functions, and overlook a foundational component, that is, the feature extractor, which is usually based on a pretrained model. This oversight limits the potential for further performance improvement. In this article, we propose a cell-wise self-optimization method to enhance the feature extractor. By leveraging the powerful representation capabilities of a pretrained model, our method further refines them for counting tasks, notably improving network performance on limited remote sensing data. Specifically, we design a lightweight cell-wise architecture optimization (CwAO) based on a network architecture search algorithm. It sequentially builds lightweight cells in parallel with the blocks in a pretrained model, leveraging a newly proposed random path selection strategy for training the optimization framework. Additionally, we propose extracting high-frequency information from the blocks in a pretrained model as self-guidance optimization to facilitate the learning of the searched cells. Experimental results on several representative datasets demonstrate that our proposed method significantly improves counting performance while substantially reducing parameters. For instance, compared with our baseline, it improves performance by 22.2% in MAE and 17.0% in RMSE on the building dataset, while reducing parameters by 61.2%.**

***Index Terms*—Knowledge distillation, lightweight neural networks, neural architecture search (NAS), remote sensing counting.**

Received 21 October 2024; revised 9 April 2025 and 22 July 2025; accepted 8 September 2025. Date of publication 15 September 2025; date of current version 29 September 2025. This work was supported in part by the Joint Funds of the National Natural Science Foundation of China under Grant U21B2044, in part by the Major Achievement Transformation Project of Jiangsu Province under Grant BA2022026, in part by the National Natural Science Foundation of China under Grant 62001237, in part by Jiangsu Planned Projects for Postdoctoral Research Funds under Grant 2021K052A, in part by China Postdoctoral Science Foundation Funded Project under Grant 2021M701756, and in part by the Startup Foundation for Introducing Talent of NUIST under Grant 2020r084. *(Corresponding authors: Qingshan Liu; Haokui Zhang.)*

Shengqin Jiang and Qian Jie are with the School of Computer Science and Jiangsu Collaborative Innovation Center of Atmospheric Environment and Equipment Technology (CICAEET), Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: jiangshengmeng@126.com; jieqian213@163.com).

Fengna Cheng is with the College of Mechanical and Electronic Engineering, Nanjing Forestry University, Nanjing 210037, China (e-mail: cfn1218@163.com).

Haokui Zhang is with the School of Cybersecurity, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: hkzhang1991@mail.nwpu.edu.cn).

Yu Liu is with Zhejiang Lab, Hangzhou 311121, China (e-mail: uniliu@zhejianglab.com).

Qingshan Liu is with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China (e-mail: qsliu@nuist.edu.cn).

Digital Object Identifier 10.1109/TGRS.2025.3608764

I. Introduction

REMOTE sensing counting estimates the number of objects of interest within a remote sensing image or video frame. This technique has diverse applications, including resource surveys, environmental monitoring, urban planning, and agricultural management. Accurate prediction in this field is challenging due to significant scale variations, cluttered backgrounds, and arbitrary orientations [1]. To address these challenges, considerable efforts have been made and yielded fruitful results. However, another challenge, namely, the limited number of training samples, is often underemphasized, which will constrain the learning capabilities of neural networks.

To alleviate this issue, a prevailing solution for counting networks is to utilize a pretrained model as the feature extractor and then pay more attention to the design of specific modules or the optimization of loss functions. For example, an attention module, a scale pyramid module, and a deformable convolution module are integrated in [1] to highlight useful features and suppress background noise. An adaptive multiscale context aggregation module is introduced to tackle the problems of large-scale variations and complex backgrounds. The learning of multiscale targets is accomplished in [2] by mining and refining scale features, and then a reliable supervised signal improved from Bayesian and counting loss is employed to learn a density distribution. The complex global regression problem is decomposed into two sub-problems [3]: learning to rank and linear transformation. This method improves both the accuracy and efficiency of the network.

The aforementioned efforts primarily focus on optimizing the network backend to enhance performance but overlook a critical component: the pretrained frontend for feature extraction. They implicitly assume that the pretrained model could adapt well to the counting task through fine-tuning. However, they neglect that there is a large domain gap between the ImageNet dataset [4] used for pretraining and the remote sensing counting dataset. This could increase the difficulty of transferring a pretrained model to the current task, thereby limiting the model's performance. Moreover, the limited training samples in the counting dataset could further exacerbate this issue. This is because the pretrained model, which typically has a large number of parameters, may not fully utilize its capacity in such cases, leading to suboptimal performance. Thus, we raise the following question: *Can we effectively adapt the pretrained model to the counting task while leveraging its powerful representation learning capabilities?*

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

To address the aforementioned question, we propose cell-wise self-optimization to better optimize the feature extractor based on a pretrained model to adapt to remote sensing counting. Note that our method still retains the use of the pretrained model, capitalizing on its powerful representation capabilities. Unlike previous methods, we specifically optimize the pretrained model at both the architecture and feature levels to better align it with the counting tasks. *At the architecture level*, we introduce cell-wise architecture optimization (CwAO) to update the parameter-heavy blocks in the pretrained model, reducing the complexity of adapting to downstream tasks by minimizing the number of parameters that require optimization. Specifically, we search for a lightweight cell as an alternative to its corresponding block in the pretrained model. Additionally, we propose a random path selection strategy for training the framework that enables the cell under search to utilize the rich pretrained knowledge. *At the feature level*, we introduce a high-frequency self-guidance optimization (HSO) method that harnesses the representational capacity of blocks in the pretrained model to enhance the expressiveness of the searched blocks. During training, this method dynamically uses high-frequency information from the output features of the corresponding block in the pretrained model to guide the learning of the newly searched cell. The goal is to improve the cell's ability to capture discriminative features in complex remote sensing scenes. Finally, extensive experiments demonstrate the effectiveness of our proposed method.

1) Unlike some existing counting works, which focus on designing the network backend, we first address the issue of weak transfer performance in the network frontend, specifically in the pretrained model used for remote sensing counting tasks. We propose a cell-wise self-optimization method that efficiently boosts the representational capabilities of the feature extractor.
2) We propose a CwAO method to optimize the parameter-heavy blocks in the pretrained model and also introduce a random path selection strategy for training the optimization framework.
3) We propose a HSO method that utilizes high-frequency features (HfFs) from the blocks in the pretrained model to help the searched cells better learn discriminative information.

II. Related Work

A. Remote Sensing Counting

In recent years, there has been significant progress in remote sensing counting. Given the limited availability of datasets, many works rely on pretrained models as feature extractors and concentrate on the design of the network backend [5]. For example, Gao et al. [1] released a large-scale RSOC dataset and then designed three modules, including an attention module, a scale pyramid module, and a deformable convolution module, to address some challenging factors in this task. Instead of using Gaussian-based density maps for supervision, some studies [2], [6] followed the approach of [7] and reformulated the counting task as a probability distribution estimation problem. Within this learning framework, multiscale features were fused through dilated convolutions with varying dilation rates, and a self-context distillation module was introduced to distill shallow feature representations from deeper ones [8]. Recently, an adaptive kernel weighting attention mechanism and a pixel-wise occupancy estimation module were proposed to mitigate numerical imbalance in Gaussian regression [9], achieving more precise density regression and improving the generalization ability of dense object counting in remote sensing images. These models typically use pretrained VGG16 or VGG19 as feature extractors. While these models excel in counting tasks, their large number of parameters can restrict their use on devices with limited computational resources. To address this issue, the lightweight MobileNet architecture [10] was adopted as the network backbone [11]. Moreover, a lightweight multiscale context fusion module and a lightweight counting-scale pooling module were proposed to effectively capture subtle target features. These designs for network backends have significantly improved counting accuracy, but they overlook another important component of the network: the feature extractor, which limits further improvement in model performance. To this end, we propose a cell-wise self-optimization method to alleviate this issue.

B. Neural Architecture Search

Neural architecture search (NAS) is an automated approach for designing neural network architectures. It seeks to identify the optimal network configuration from a wide range of candidates, aiming to achieve the best performance while maintaining a compact parameter number. Various NAS methods have been proposed, including stochastic search, evolutionary search, and gradient-based methods. The gradient-based methods therein, such as DARTS [12], offer higher search efficiency and faster convergence. Consequently, considerable studies are dedicated to improving this technique. To address the memory and computational overhead issues in DARTS, PC-Darts [13] sampled only a portion of the supernet's channels at each step, thereby reducing redundancy in the search space and achieving a more efficient search without compromising performance. Building on this method, EPC-Darts [14] introduced an efficient channel attention mechanism based on partial channel connection to solve the problems of low memory utilization efficiency and poor search architecture stability. At the same year, Zhang et al. [15] theoretically found that using random features can help mitigate performance collapse by reducing the impact of skip connections. However, most existing DARTS methods typically train the supernet from scratch, preventing them from leveraging the rich semantic knowledge available in pretrained models. Additionally, these methods primarily focus on classification tasks and do not specifically address remote sensing counting tasks. To this end, we search for efficient cells to take over the blocks in pretrained models. These cells aim to both harness the strong representation learning capabilities of the pretrained model and reduce its computational complexity.

C. Knowledge Distillation

In general, knowledge distillation focuses on transferring valuable knowledge from a teacher model to a lightweight

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

student model [16]. For instance, neural architecture knowledge from a teacher model was distilled to guide block-wise architecture search [17], thereby improving the effectiveness of NAS. Meanwhile, three types of knowledge from the teacher model, namely, interlayer, intralayer, and response knowledge, were explored to regularize the learning process of the student model [18]. Afterward, the activation maps of each channel in both the teacher and student branches were normalized in [19] and then aligned using KL divergence, enabling the student network to more effectively learn the salient features of the teacher in dense prediction tasks. The semantic similarity metric and relational similarity metric were used to enhance the performance of distillation in [20]. The aforementioned methods follow the teacher-to-student learning paradigm, which involves two-stage training and thus requires additional computational time.

Although this paradigm yields a lightweight student model, its performance is typically constrained by the capabilities of the teacher model. To this end, some works have turned to the self-distillation of features within the model to improve the performance of the model. In [21], the knowledge in the deeper portion of the networks was guided to the shallow ones. MaskCLIP [22] distilled representation from a full image to the representation predicted from a masked image. Wu et al. [23] introduced two spatial self-distillation strategies to extract spatial information with the goal of refining inaccurate bounding boxes. These self-distillation approaches primarily focus on transferring knowledge within homogeneous network structures, either across different feature levels or inputs, but they overlook the potential of extracting valuable knowledge between heterogeneous structures. In this work, we propose a high-frequency knowledge distillation strategy tailored for remote sensing counting. It enables the searched cells to learn the rich semantic information from the corresponding blocks in pretrained models.

III. Method

In this section, we introduce a cell-wise self-optimization strategy for remote sensing counting. This strategy includes CwAO and HSO. We will start with an overview of the overall network architecture and then discuss each optimization strategy in detail.

A. Method Overview

Fig. 1 shows the overview of the proposed network architecture for remote sensing counting. It comprises an encoder, based on a pretrained network, and a decoder. For brevity, the pretrained encoder $D(\cdot)$ is divided into a BaseNet $g(\cdot)$ and N stacked blocks (i.e., $f_i(\cdot)$, $i = 1, 2, \ldots, N$). The BaseNet extracts shallow features from the visual input, while these blocks capture mid-level and high-level semantic representations. Any given input $x \in \mathcal{X}$ is mapped to the feature space $\mathcal{Y}$ after passing through the encoder, which is formulated as follows:

$$D(x) = f_{1\to N} \circ g(x) \tag{1}$$

where $f_{1\to N} = f_N \circ f_{N-1} \circ \cdots \circ f_1$, and $\circ$ denotes the composite operation.

On top of the two network components, we propose a CwAO strategy. It aims to replace each block by searching for an optimal cell in parallel, thus making the feature extractor more suited to the specific task. Additionally, to better utilize the inherent semantic knowledge of pretrained networks, we further propose a simple and effective HSO strategy. It helps the newly searched lightweight cells learn key representational information from counting datasets.

B. Cell-Wise Architecture Optimization

Pretrained models, usually trained on a large-scale dataset, can expedite the adaptation of neural networks to remote sensing counting tasks. However, due to the large domain gap between ImageNet [4] and remote sensing image datasets, along with the limited sample number of remote sensing counting datasets, this may lead to performance bias in pretrained models when applied to the new task. To address this issue, we develop the CwAO that searches for a lightweight cell to replace the corresponding block in the pretrained model. This approach aims to provide fresh insights from the newly searched cells for the counting task while leveraging the strengths of the pretrained model, thus strengthening the model's understanding of the downstream task. Additionally, it can reduce the heavy parameters of the blocks, resulting in a lighter model that is easier to deploy on devices with limited computing resources. In what follows, we will introduce how to optimize the architecture of pretrained models.

As the depth of the pretrained model increases, it can capture context information over a broader range, allowing it to represent more complex patterns and structures. This makes the choice of which block to optimize challenging. To this end, we build N parallel searchable cells $(p_i, i = 1, 2, \ldots, N)$ aligned with each block in the pretrained model. Unlike traditional architecture search algorithms where cells are stacked sequentially, our cell is independent and designed to effectively replace the corresponding block in pretrained models. To better retain the representational capabilities of original pretrained models while improving the adaptability to counting data, we introduce a simple yet effective random path selection strategy. This involves randomly selecting a cell to replace the corresponding block in each feed-forward process. This process is formulated as follows:

$$\vec{D}(x) = f_{m+1\to N} \circ p_m \circ f_{1\to m-1} \circ g(x) \tag{2}$$

where $\vec{D}(x)$ denotes the newly constructed encoder, and $m = \text{Randsampling}(1, N)$.

In pretrained models, it is common practice to increase the number of channels after reducing the spatial resolution of feature maps in order to enhance the learning capacity of networks. However, it can result in a significant increase in the number of network parameters, particularly in the deeper layers of the network. This makes it impractical for devices with limited computing resources. Therefore, each cell we search should be as simple and lightweight as possible. To achieve this, we utilize two separate 3×3 convolutions to transform the channel dimension of features before and after each cell and set a small number of channels for each

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

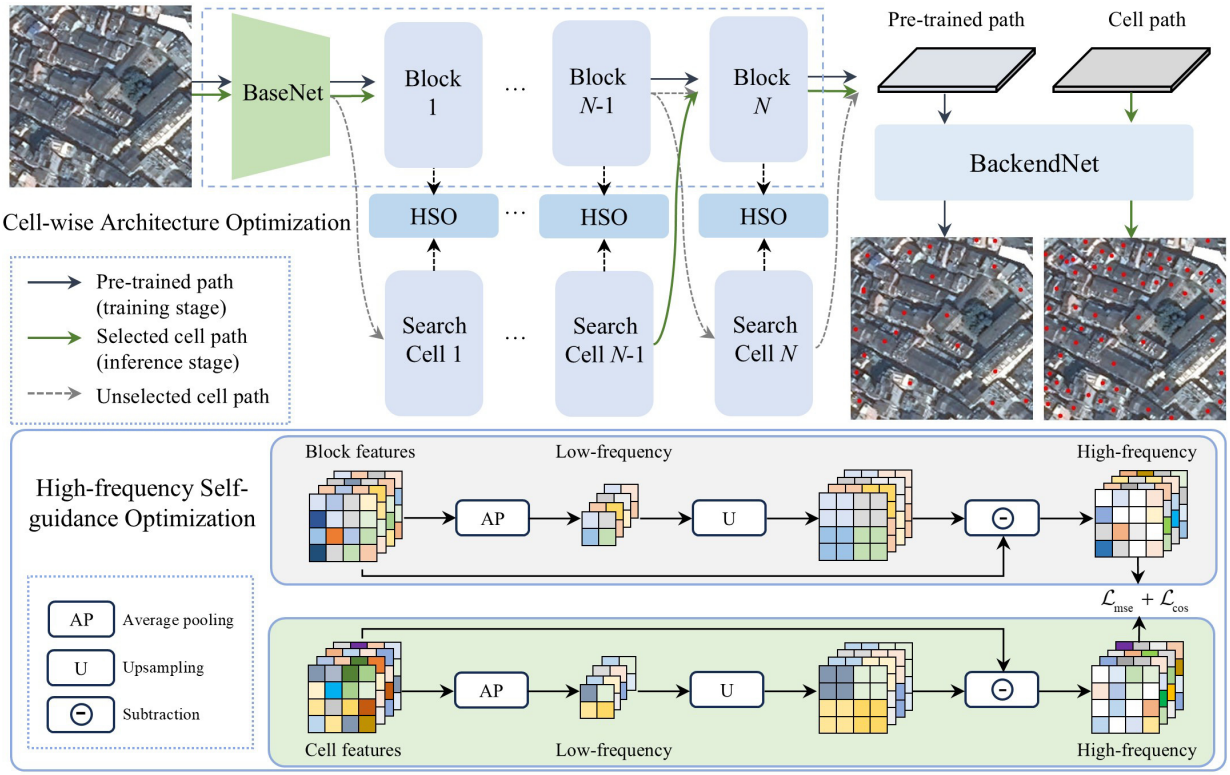


Fig. 1. Overview of our proposed network architecture. We first propose a cell-wise self-optimization method to better optimize the feature extractor based on a pretrained model to adapt to remote sensing counting. At the architecture level, we propose a CwAO method that replaces the parameter-heavy blocks in a pretrained model with optimized cells, making the network lightweight while enabling rapid adaptation to the complex remote sensing scene. During the training phase, we randomly replace the corresponding block in the pretrained model with a searched cell while simultaneously executing the training of the pretrained path in parallel. At the feature level, we introduce a HSO method that leverages the prior features of the pretrained block to guide the learning of the corresponding cell, thereby enhancing its representational capabilities.

convolution operation within the cell. In our design, each cell comprises two layers, with each layer containing three nodes. In the search space, we define four different operations

1) con_3_1 : ReLU − Conv2D (kernel size = 3, dilation = 1) − BN
2) con_3_2 : ReLU − Conv2D (kernel size = 3, dilation = 2) − BN
3) con_3_3 : ReLU − Conv2D (kernel size = 3, dilation = 3) − BN
4) zero: $f(x) = 0$

where ReLU and BN indicate the ReLU activation function and batch normalization, respectively. Conv2D (kernel size = x, dilation = y) denotes a 2-D convolution with a kernel size of $x \times x$ and a dilation rate of y.

C. High-Frequency Self-Guidance Optimization

To better leverage the knowledge inherent in the pretrained model, we introduce high-frequency guided feature optimization. This strategy aims to use the features from the replaced block in the pretrained model to guide the learning of its corresponding cell during training. During the training phase, we set up two parallel paths: one based on the pretrained model (referred to as the pretrained feature path, see 1) and one based on the searched cell (referred to as the cell feature path, see 2). These paths share BaseNet and the remaining blocks in the pretrained model, thereby avoiding additional computational overheads.

To facilitate knowledge transfer between both paths, we propose using HfFs. It is because in complex environments, weak targets often blend with background information due to low contrast, making them difficult to distinguish. This challenge is particularly evident in remote sensing images, where targets are typically surrounded by intricate backgrounds. For instance, vehicles may have colors similar to roads, while buildings may share textures with the ground, complicating target separation for traditional methods. HfFs effectively capture edge information, sharpening target contours and improving contrast between targets and their background. Specifically, for any given feature F_m, we first apply an average pooling layer to extract low-frequency information $F_{m,lw}$, as the averaging operation acts as a low-pass filter [24]. We then subtract this low-frequency information from the original feature (OF) after upsampling to obtain the HfF $F_{m,hg}$. Formally, these steps can be expressed as follows:

$$\begin{cases} F_{m,lw} = \text{Avgpooling}\left(F_m\right) \\ F_{m,hg} = F_m - \text{Up}\left(F_{m,lw}\right) \end{cases} \tag{3}$$

where Avgpooling$(\cdot)$ and Up$(\cdot)$ denote the average pooling and upsampling operations, respectively.

We utilize the mean squared error loss $\mathcal{L}_{\text{mse}}$ and cosine similarity loss $\mathcal{L}_{\cos}$ to enhance the network's ability to perceive

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

high-frequency information through pixel-level and structural features. The specifics of each loss function are as follows:

$$\mathcal{L}_{\text{mse}} = \left\| F_{f_m,hg}(x) - F_{p_m,hg}(x) \right\|^2 \tag{4}$$

and

$$\mathcal{L}_{\cos} = \frac{F_{f_m,hg}(x) \cdot F_{p_m,hg}(x)}{\left| F_{f_m,hg}(x) \right| \cdot \left| F_{p_m,hg}(x) \right|} \tag{5}$$

where $F_{f_m,hg}(\cdot)$ and $F_{p_m,hg}(\cdot)$ denote the HfFs of the pth block in pretrained model and the corresponding cell, respectively.

D. Loss Function

Following the work [25], we employ the Euclidean loss $\mathcal{L}_{\text{loc}}$ to supervise the point regression, and use the cross entropy loss $\mathcal{L}_{\text{cls}}$ to supervise the proposal classification. The total loss can be calculated as

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cls}} + \lambda_1 \mathcal{L}_{\text{loc}} + \lambda_2 \mathcal{L}_{\text{mse}} + \lambda_3 \mathcal{L}_{\cos} \tag{6}$$

where λ_1, λ_2, and λ_3 are hyperparameters, and the losses $\mathcal{L}_{\text{loc}}$ and $\mathcal{L}_{\text{cls}}$ are defined as follows:

$$\mathcal{L}_{\text{cls}} = -\frac{1}{Q}\left(\sum_{i=1}^{K} \log \widehat{c}(i) + \lambda_0 \sum_{i=K+1}^{Q} \log\left(1 - \widehat{c}(i)\right) \right) \tag{7}$$

and

$$\mathcal{L}_{\text{loc}} = \frac{1}{K} \sum_{i=1}^{K} \left\| p(i) - \widehat{p}(i) \right\|^2 \tag{8}$$

where K and Q denote the total number of labeled points and the number of candidate points predicted by the network, respectively, $\widehat{c}(i)$ is the confidence score of the network, λ_0 is a hyperparameter, and $p(i)$ and $\widehat{p}(i)$ denote the labeled point and predicted point of the network, respectively.

IV. Experiments

A. Datasets and Evaluation Metrics

To evaluate the validity of our method, we evaluate our model on the two challenging benchmarks: the RSOC dataset [1] and the STAR dataset [26]. Following [1], RSOC contains four categories for remote sensing counting: buildings, small vehicles, large vehicles, and ships. This dataset comprises 3057 images with 2 86 539 annotations in total. Specifically, the building subset consists of 1205 training images and 1263 test images, totaling 2468 images. Apart from the building subset, the other three categories are sourced from the DOTA dataset [27]. The small-vehicle subset includes 280 images of small vehicles, with 200 used for training, 22 for validation, and 58 for test. The large-vehicle subset consists of 172 images of large vehicles, with 98 used for training, ten for validation, and 64 for test. Additionally, the ship subset contains 137 images, with 88 used for training, nine for validation, and 40 for test.

STAR [26] provides large-scale, high-resolution satellite imagery with over 2 10 000 annotations covering a wide range of complex scenarios. To evaluate the performance of our method, we adopt five representative categories [28]: airplanes, wind turbines, lattice towers, bridges, and runways. The dataset is partitioned as follows: airplanes (233 training images/106 validation images), wind turbines (29 training/11 validation), lattice towers (132 training/39 validation), bridges (198 training/43 validation), and runways (239 training/106 validation). Notably, the test data is evaluated exclusively through an online server. However, since the ground truth annotations are provided in bounding box format, they are incompatible with the point-based predictions of our method. Consequently, quantitative evaluations are performed solely on the validation split.

Here, we employ MAE and RMSE as evaluation metrics. MAE is used to evaluate the accuracy of the model, while RMSE is used to measure the robustness and stability of the model. They are defined as

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} \left| d_i - \widehat{d}_i \right| \tag{9}$$

and

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \left(d_i - \widehat{d}_i \right)^2} \tag{10}$$

where d_i and $\widehat{d}_i$ represent the ground truth and predicted result of the ith image, respectively, and N represents the total number of test images. Additionally, we use the number of network parameters (Param.), floating point operations per second (FLOPs), and frames per second (FPS) to measure the computational complexity of models.

B. Implementation Details

During the search phase, we closely follow the setup of DARTS [12]. We split the training dataset into two parts: one for optimizing cell architecture parameters and the other for optimizing cell network weights. Unless otherwise specified, all of our models in the experiment adopt the framework of P2PNet, which uses the first ten layers of the pretrained VGG16 as a backbone. We set the first four layers as BaseNet and divide the subsequent six layers into two blocks. Following [25], we set the hyperparameters λ_0 and λ_1 to 0.5 and $2e{-}4$, respectively. The values of λ_2 and λ_3 are set to $1e{-}6$ and $1e{-}5$, respectively. The Adam optimizer is used for network optimization. The learning rates for the pretrained model, cell architecture parameters, and cell network weights are set to $1e{-}5$, 0.1, and $1e{-}3$, respectively, with a batch size of 8. During the training phase, the full training dataset is utilized. We employ the same network optimizer and learning rates as in the search phase. Additionally, we implement data augmentation techniques such as random scaling, random cropping, and horizontal flipping to enhance network learning.

C. Compared With SOTA Methods

In Table I, we report the comparison results with state-of-the-art (SOTA) methods. It is evident that our method achieves superior performance across almost all metrics with a lower number of parameters. Specifically, P2PNet [25] can be considered a strong baseline, leveraging the first 13 layers of the pretrained VGG16 as its feature extractor. In comparison, our

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

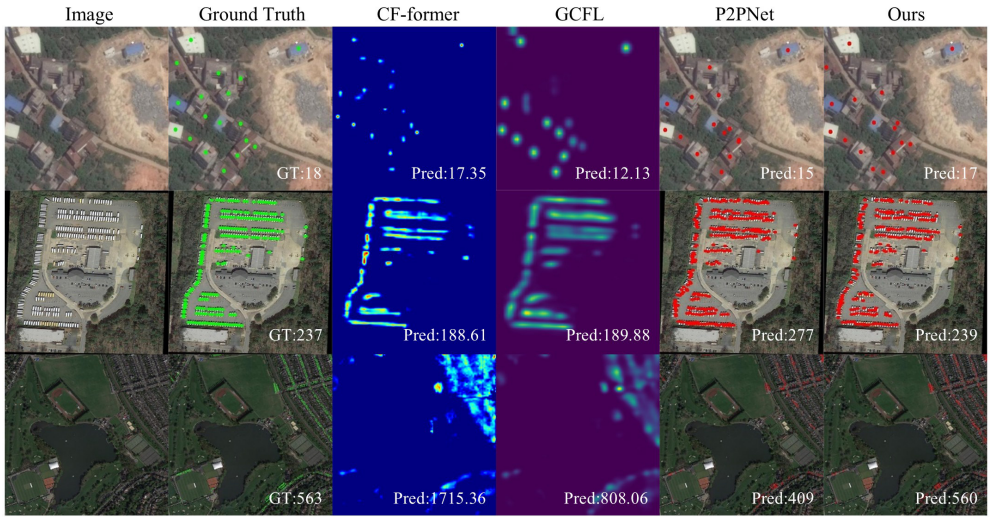


Fig. 2. Visualization of prediction results compared to other methods.

TABLE I
COMPARISON WITH SOME SOTA METHODS ON RSOC. THE BEST AND SECOND BEST RESULTS ARE MARKED IN BOLD AND UNDERLINE, RESPECTIVELY

Method	Param.(M)	FLOPs(G)	FPS	Building		Small-vehicle		Large-vehicle		Ship	
				MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
Bayesian [7]	21.50	26.99	10	28.95	32.93	185.79	703.31	26.03	51.54	64.15	86.01
ASPD [1]	22.70	37.96	4	7.59	10.66	-	-	42.87	63.49	216.27	352.23
P2PNet [25]	19.2	23.29	10	7.46	10.34	174.00	666.44	11.61	16.94	62.00	76.43
MAN [6]	40.41	27.13	8	29.01	32.98	505.10	1326.12	62.78	79.65	**47.2**	70.24
PET [29]	20.91	28.42	5	10.78	13.96	419.74	1212.18	36.39	54.52	258.83	396.39
LMSFFNet [11]	4.58	14.90	-	6.52	9.67	141.7	**273.0**	12.74	27.13	49.47	70.0
GCFL [30]	21.51	27.01	10	7.79	10.74	222.40	768.83	16.30	23.31	55.32	77.05
PML [31]	16.26	21.65	8	8.3	12.78	217.17	652.98	32.15	48.2	139.58	172.17
BDRNet [9]	17.09	20.87	7	13.94	18.70	508.58	1297.17	52.08	75.21	179.79	264.17
CF-former [5]	110.19	29.18	4	6.97	10.61	204.55	634.60	18.95	34.01	51.01	94.57
Ours	3.49	13.40	17	**6.42**	**9.23**	**125.64**	416.56	**9.45**	**15.77**	49.28	**67.86**

method consistently outperforms P2PNet across four datasets while reducing the number of parameters by 81.8%. For example, on the small-vehicle subset, our method improves MAE and RMSE by 27.8% and 37.5%, respectively. On ship, our method also improves by 20.5% and 11.2% on the two metrics, respectively. This indicates that our proposed framework can significantly improve the computational complexity and network performance by using a cell-level search method to update and replace the block in the pretrained model. LMSFFNet [11] is a typical representative of lightweight networks, which selects MobileViT as the backbone and empirically designs multiscale fusion and refinement strategies to better characterize the targets of interest. Compared to LMSFFNet, our method surpasses it on all datasets with 23.8% fewer parameters. This demonstrates the effectiveness of our strategy. We also visualize the predicted results of our method with some SOTA methods in Fig. 2. It is evident that our method performs well across different density levels. This further demonstrates the effectiveness and adaptability of our method in various scenarios.

To further validate the effectiveness of our method, we present a comparison with other methods in Table II. Our method clearly outperforms others in terms of average metrics across multiple categories. Notably, it achieves this with significantly fewer parameters. Specifically, compared to P2PNet, which ranks second in performance, our method shows substantial improvements across various categories while utilizing less than one-fifth of P2PNet's parameters. This demonstrates not only the effectiveness of our method but also its scalability for learning additional object categories.

To demonstrate the efficiency of our method, we deploy it on the Jetson Nano TX2 edge device to assess its inference speed. The results, presented in Tables I and II, show that our method achieves 17 frames/s, significantly outperforming other models while preserving optimal performance across multiple categories. Notably, our method is 1.7 times faster than the strong baseline, P2PNet, while requiring significantly fewer parameters. It should be emphasized that these tests are conducted without any model deployment optimizations.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

TABLE II

COMPARISON WITH SOME SOTA METHODS ON STAR. THE BEST AND SECOND BEST RESULTS ARE MARKED IN BOLD AND UNDERLINE, RESPECTIVELY

Method	Param.(M)	FLOPs(G)	FPS	Airplane		Wind mill		Lattice tower		Bridge		Runway		Average	
				MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
MAN [6]	40.41	27.13	8	46.82	66.68	19.73	25.88	26.54	36.43	3.02	5.96	2.06	2.31	19.63	27.45
Bayesian [7]	21.50	26.99	10	45.94	66.08	18.94	25.22	25.93	36.00	2.28	5.20	1.39	1.76	18.90	26.85
PML [31]	16.26	21.65	8	65.66	77.98	2.68	9.26	12.15	24.00	3.63	5.38	5.46	5.94	17.92	24.51
PET [29]	20.91	28.42	5	39.86	59.51	14.82	22.26	22.38	33.52	2.56	5.44	1.98	2.28	16.32	24.60
BDRNet [9]	17.09	20.87	7	18.12	27.26	8.20	15.63	18.67	26.54	2.14	5.42	1.09	1.35	9.64	15.24
GCFL [30]	21.51	27.01	10	14.18	23.37	2.42	3.42	17.96	28.41	2.10	5.39	**0.82**	**1.11**	7.50	12.34
P2PNet [25]	19.2	23.29	10	**12.20**	21.92	**0.45**	**0.67**	12.21	21.05	1.60	2.46	1.18	1.51	5.53	9.52
Ours	3.49	13.40	17	12.81	**20.16**	**0.45**	**0.67**	**10.15**	**15.98**	**1.47**	**1.96**	1.14	1.44	**5.20**	**8.04**

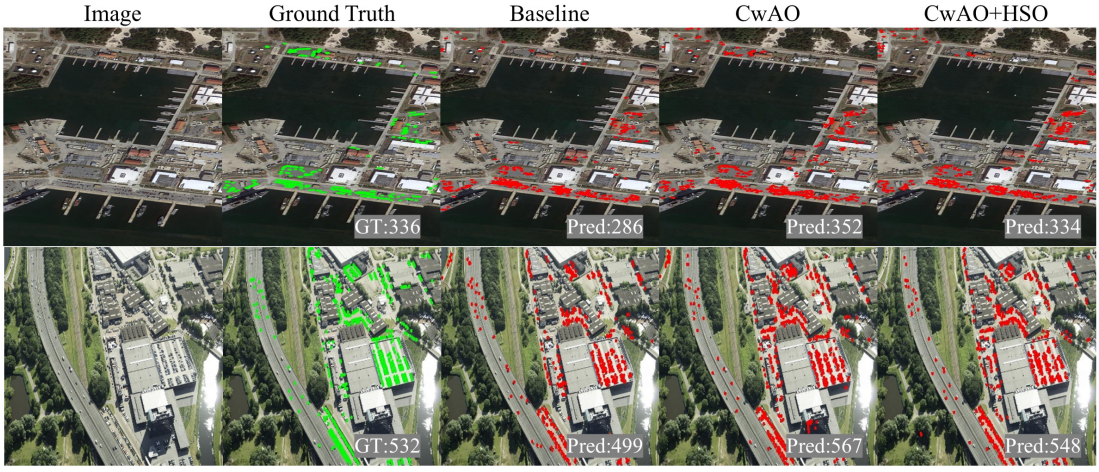


Fig. 3. Visualization of key components in our proposed method.

TABLE III

ABLATION STUDY ON OUR PROPOSED KEY COMPONENTS. CWAO AND HSO DENOTE CELL-WISE ARCHITECTURE OPTIMIZATION AND HIGH-FREQUENCY SELF-GUIDANCE OPTIMIZATION, RESPECTIVELY

Method	Param.(M)	FLOPs(G)	MAE	RMSE
Baseline	8.99	19.63	8.25	11.12
Baseline+CwAO	3.49	13.40	7.15	10.05
Baseline+CwAO+HSO	3.49	13.40	**6.42**	**9.23**

D. Ablation Study

All the ablation studies are conducted on the building subset to verify the effectiveness of our method.

1) Effectiveness of Key Components: In Table III, we conduct ablation studies by incrementally introducing key components of our proposed method. The first row denotes our baseline, which is a parameter-reduced version of P2PNet. It is built upon VGG16 features at 1/8 resolution, along with reduced channel numbers in the classification and regression heads. When introducing CwAO, the performance improves to 7.15 in MAE and 10.05 in RMSE, with at least a 13.3% (9.6%) performance boost. Meanwhile, this method reduces the number of parameters by 61.2% and FLOPs by 31.7%. Further adding HSO, the performance is improved to 6.42 in MAE and 9.23 in RMSE without increasing any parameters. These results indicate that our method can significantly improve the performance of the pretrained model in remote sensing counting tasks while substantially reducing the computational complexity of the model.

We further highlight the effectiveness of the key components in our method through visualization results. As illustrated in Fig. 3, the introduction of CwAO enhances the ability of our network to identify difficult targets, leading to a significant improvement in counting performance. Moreover, incorporating HSO brings the network's predictions closer to the actual values. The results further confirm the effectiveness of our proposed strategies.

2) Comparison of Inference Paths of Different Pretrained Models: Since searching for different blocks in the pretrained model yields distinct cells, this enables us to obtain varying inference paths. For this reason, we explore the impact of these paths on network performance, and we also discuss the effect of different pretrained models on these results, as shown in Table IV. Our findings are as follows.

1) Under different pretrained models, replacing any block with the searched cell results in a clear improvement in network performance with a reduced number of parameters compared to the baseline. This suggests that our proposed strategy can effectively enhance the transferability of pretrained models.
2) Replacing higher level blocks in the pretrained model can bring about substantial performance improvements while also significantly reducing the number of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

ABLATION STUDY ON INFERENCE PATHS OF DIFFERENT PRETRAINED MODELS. RB DENOTES A SPECIFIC BLOCK THAT IS BEING REPLACED IN THE PRETRAINED MODEL USING A SEARCHED CORRESPONDING CELL, "–" INDICATES OUR BASELINE THAT IS OBTAINED THROUGH FINE-TUNING A PRETRAINED MODEL

Backbone	RB	Param.(M)	FLOPs(G)	MAE	RMSE
VGG19	–	11.94	24.45	9.39	12.77
	1st-block	10.16	17.16	8.28	11.83
	2nd-block	4.07	16.40	7.74	11.24
VGG16	–	8.99	19.63	8.25	11.12
	1st-block	7.85	14.76	7.78	11.03
	2nd-block	3.49	13.40	**6.42**	**9.23**

TABLE V

ABLATION STUDY ON DIFFERENT SEARCH STRATEGIES. DAS-S, DAS-M, AND DAS-L DENOTE THE SMALL-, MEDIUM-, AND LARGE-PARAMETER VERSIONS OF THE SEARCHED MODELS BY USING THE DAS ALGORITHM, RESPECTIVELY. BASEDAS REFERS TO DIRECTLY REPLACING THE FINAL BLOCK IN THE PRETRAINED MODEL WITH A SEARCHED CELL

Method	Param.(M)	FLOPs(G)	MAE	RMSE
Baseline	8.99	19.63	8.25	11.12
DAS-S	1.73	6.65	8.89	12.80
DAS-M	2.46	19.53	8.6	12.11
DAS-L	3.54	40.02	8.41	12.19
BaseDAS	3.49	13.40	12.46	15.81
Ours	3.49	13.40	**6.42**	**9.23**

TABLE VI

ABLATION STUDY ON FEATURE OPTIMIZATION STRATEGIES. "OFS," "LFFS," AND "HFFS" DENOTE THE ORIGINAL FEATURES, LOW-FREQUENCY FEATURES, AND HIGH-FREQUENCY FEATURES, RESPECTIVELY

OFs	LfFs	HfFs	MAE	RMSE
✓			7.42	10.20
	✓		10.20	16.55
		✓	**6.42**	**9.23**
	✓	✓	6.99	9.82

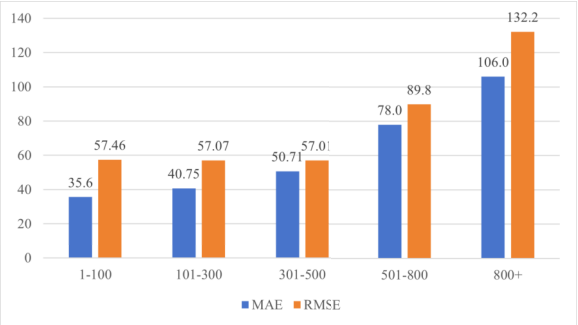


Fig. 4. Comparison results of our method under different density distributions on the ship subset.

parameters. This indicates that while higher level blocks can extract more abstract semantic features, they may also present an impediment that potentially affects the model's transferability.

3) Despite the great representational power of VGG19, its performance is inferior to that of VGG16-based models across all our experimental setups. We hypothesize that this is due to the limited samples in this task, which increases the difficulty of fine-tuning parameter-heavy models. Therefore, we choose VGG16 as our backbone and opt to replace the last block in the pretrained model during the inference phase to achieve optimal performance.

3) Effects of Search Strategies: We further investigate the effects of different search strategies on network performance, as shown in Table V. To achieve this, we construct pure differentiable architecture search (DAS) models trained from scratch. They are implemented by stacking four searchable cells sequentially, with a search space consistent with our method. There are small, medium, and large versions by changing the channel numbers of the searching networks. It can be observed that the models obtained through DAS, despite having varying numbers of network parameters, show slightly improved but comparable performance. These models perform even worse than our baseline, particularly in terms of RMSE. This suggests that the pretrained model can enhance the model's stability and adaptability in remote sensing counting tasks. Further, we explore whether the performance gain of our method comes from directly replacing blocks in the pretrained model with searched cells. The experimental results in the fourth row demonstrate that this leads to a degradation in network performance. We speculate that this is due to the inability to leverage higher level semantic information from the pretrained model, thus hindering efficient transfer learning. This indicates that the gain of our method comes from the strategy we design rather than from a single pretrained model or architecture search. In other words, the performance improvement is achieved by optimizing the network architecture and features while leveraging the advantages of pretrained models.

4) Effects of Feature Optimization Strategies: HfFs play a crucial role in guiding the learning of searched cells. For this reason, we further explore the comparison of these features against other guiding features, such as the OFs and low-frequency features (LfFs), in Table VI. As shown, the performance using LfFs is clearly weaker than that obtained with OFs. This might be due to the lack of detailed information in LfFs, making it difficult for the model to distinguish similar targets. HfFs, on the other hand, aid in the recognition of detailed structures or boundaries within scenes, which is essential for discerning challenging targets. Their performance improves by 13.5% in MAE and 9.5% in RMSE compared to the results obtained with OFs. We also combine HfFs and LfFs, but the results are slightly inferior to those achieved with HfFs alone. It could be due to LfFs introducing some noise into the transferred knowledge. In sum, these results demonstrate the effectiveness of our HSO.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

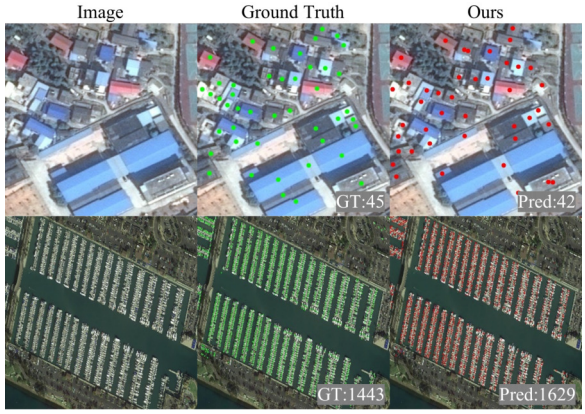


Fig. 5. Failure cases of our method.

E. Limitations

To provide a clearer understanding of our method, we present its limitations in two aspects. The first limitation concerns the difference in prediction performance across different density intervals. As shown in Fig. 4, our method performs relatively well in lower density regions but exhibits a noticeable decline in accuracy as target density increases. This decline may stem from two factors: 1) higher target density reduces visual distinguishability between objects, making them harder to detect and 2) substantial variation in data distribution across different density levels in the training set hinders the model's ability to learn balanced predictions across distributions.

The second limitation is the failure cases of our method, as illustrated in Fig. 5. In the first row, although the predicted results are relatively close to the ground truth, false negatives and false positives are evident. These errors likely arise from significant variations in target scales and the high similarity between background and foreground, which make it challenging to distinguish targets of interest. In the second row, the proposed method significantly overestimates the number of targets. This issue may be attributed to extreme crowding in the scene, which complicates the model's ability to perceive individual objects accurately.

V. Conclusion

This article introduces a cell-wise self-optimization strategy to improve pretrained models for remote sensing counting. Compared to direct fine-tuning methods, our method achieves a significant performance boost while drastically reducing the model's parameters. To achieve this, we employ an architecture search algorithm to find lightweight cells that can replace their corresponding blocks in the pretrained model and also propose a random path selection method for training this framework. The proposed method leverages the benefits of the pretrained model while enhancing its adaptability to the current task. Furthermore, we utilize HfFs to guide the learning of the searched cell, improving its representation capability. Finally, extensive experiments validate the effectiveness of our method and demonstrate SOTA performance. We hope that this optimization strategy can be extended to more downstream tasks.

References

[1] G. Gao, Q. Liu, and Y. Wang, "Counting from sky: A large-scale data set for remote sensing object counting and a benchmark method," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 3642–3655, May 2021.

[2] G. Gao, Q. Liu, Z. Hu, L. Li, Q. Wen, and Y. Wang, "PSGCNet: A pyramidal scale and global context guided network for dense object counting in remote-sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5619412.

[3] Y. Huang, Y. Jin, L. Zhang, and Y. Liu, "Remote sensing object counting through regression ensembles and learning to rank," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4701617.

[4] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.

[5] S. Jiang, J. Cai, H. Zhang, Y. Liu, and Q. Liu, "Compare and focus: Multi-scale view aggregation for crowd counting," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 10, pp. 13231–13239, Oct. 2024.

[6] H. Lin, Z. Ma, R. Ji, Y. Wang, and X. Hong, "Boosting crowd counting via multifaceted attention," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 19628–19637.

[7] Z. Ma, X. Wei, X. Hong, and Y. Gong, "Bayesian loss for crowd count estimation with point supervision," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2019, pp. 6142–6151.

[8] Z. Duan, S. Wang, H. Di, and J. Deng, "Distillation remote sensing object counting via multi-scale context feature aggregation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–12, 2022.

[9] H. Guo, J. Gao, and Y. Yuan, "Balanced density regression network for remote sensing object counting," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5625013.

[10] S. Mehta and M. Rastegari, "Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–26.

[11] J. Yi, Z. Shen, F. Chen, Y. Zhao, S. Xiao, and W. Zhou, "A lightweight multiscale feature fusion network for remote sensing object counting," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5902113.

[12] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jun. 2018, pp. 1–12.

[13] Y. Xu et al., "PC-DARTS: Partial channel connections for memory-efficient architecture search," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–13.

[14] Z. Cai, L. Chen, and H.-L. Liu, "EPC-DARTS: Efficient partial channel connection for differentiable architecture search," *Neural Netw.*, vol. 166, pp. 344–353, Sep. 2023.

[15] X. Zhang, Y. Li, X. Zhang, Y. Wang, and J. Sun, "Differentiable architecture search with random features," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 16060–16069.

[16] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[17] C. Li et al., "Block-wisely supervised neural architecture search with knowledge distillation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1989–1998.

[18] L. Liu, J. Chen, H. Wu, T. Chen, G. Li, and L. Lin, "Efficient crowd counting via structured knowledge transfer," in *Proc. 28th ACM Int. Conf. Multimedia*, 2020, pp. 2645–2654.

[19] C. Shu, Y. Liu, J. Gao, Z. Yan, and C. Shen, "Channel-wise knowledge distillation for dense prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5311–5320.

[20] P. Dong, L. Li, and Z. Wei, "DisWOT: Student architecture search for distillation WithOut training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 11898–11908.

[21] L. Zhang, J. Song, A. Gao, J. Chen, C. Bao, and K. Ma, "Be your own teacher: Improve the performance of convolutional neural networks via self distillation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 3713–3722.

[22] X. Dong et al., "MaskCLIP: Masked self-distillation advances contrastive language-image pretraining," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 10995–11005.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.

[23] D. Wu, P. Chen, X. Yu, G. Li, Z. Han, and J. Jiao, "Spatial self-distillation for object detection with inaccurate bounding boxes," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 6832–6842.

[24] E. Voigtman and J. D. Winefordner, "Low-pass filters for signal averaging," *Rev. Sci. Instrum.*, vol. 57, no. 5, pp. 957–966, May 1986.

[25] Q. Song et al., "Rethinking counting and localization in crowds: A purely point-based framework," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 3365–3374.

[26] Y. Li et al., "STAR: A first-ever dataset and a large-scale benchmark for scene graph generation in large-size satellite imagery," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 1832–1849, Mar. 2025.

[27] G.-S. Xia et al., "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3974–3983.

[28] S. Jiang, Y. Gao, B. Li, F. Cheng, R. Hang, and Q. Liu, "Remote sensing object counting with online knowledge learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5615511.

[29] C. Liu, H. Lu, Z. Cao, and T. Liu, "Point-query quadtree for crowd counting, localization, and more," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 1676–1685.

[30] W. Shu, J. Wan, and A. B. Chan, "Generalized characteristic function loss for crowd analysis in the frequency domain," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 2882–2899, May 2024.

[31] Z. Yan et al., "Progressive multi-resolution loss for crowd counting," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 5, pp. 3232–3244, May 2024.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 11:53:00 UTC from IEEE Xplore. Restrictions apply.